



CLASSIFICATION OF DEGREE OF FRESHNESS AND
HEALTHINESS OF FRUITS AND VEGETABLES WITH
A CASE STUDY OF TOMATOES

by

OBADAH NIDAL GHIZAWI, 121200038
HAMZA SALLAM, 120200013
ABDULMOEZ S. DERRIJA, 120200136

Supervised by

ELENA BATTINI SÖNMEZ

Submitted to the

FACULTY OF ENGINEERING AND NATURAL SCIENCES

in partial fulfillment of the requirements for the

Bachelor of Science

in the

DEPARTMENT OF COMPUTER ENGINEERING

June 4, 2024

Abstract

*To reduce food waste and maximize the nutritional value of fruits and vegetables, freshness and healthiness levels must be considered. Existing approaches to assess the level of freshness go one of two routes: either consider a binary class problem where known fruits/vegetables are classified into two classes: fresh or rotten, or three classes categorization: pure-fresh, medium-fresh, or rotten. The research paper of Fahad et al. [1] focuses on the three-class approach, implementing a VGG16 model to solve this problem. The initial aim of this thesis is to challenge the paper of Fahad et al. by implementing the same multi-class approach. We will use the same dataset utilized by the approach as mentioned earlier which includes 60,000 images of 11 fruits and vegetables divided into three classes of freshness. The classification and categorization of fruits and vegetables will be performed utilizing the VGG16 architecture. The benchmark paper achieved an accuracy of 82% with the VGG16 model and 84% with YOLO v5. After going through many experiments, **we reached 94.58% test accuracy with VGG16, improving the accuracy of the benchmark paper by 11 percentage points.** The second aim of this thesis is to detect the healthiness of fruits and vegetables. We focused on the issue of detecting diseases on tomato leaves, which plays a major role in agricultural sustainability and food safety. Tomatoes are the second most important fruit or vegetable crop after potatoes. Almost 90% of edible tomatoes are thrown away, making them a significant contributor to overall fruit and vegetable waste. Like the benchmark paper [2], we have implemented 6 different CNN models to classify 10 different diseases of tomatoes using a subset of the public PlantVillage dataset and picked **EfficientNet-B0** as the best model. **Currently, we achieved a 96.04% test accuracy.** Future work includes working on data augmentation to improve this initial performance.*

TABLE OF CONTENTS

Abstract	ii
Table of Contents	iii
List of Figures	v
List of Tables	vi
List of Abbreviations	vii
1 Introduction	1
2 Related Works	3
3 Project Design	7
4 Dataset	8
4.1 FruitVeg Dataset	8
4.2 PlantVillage Dataset	9
5 Methodology	12
5.1 Neural Networks	12
5.2 Convolutional Neural Networks (CNN)	12
5.2.1 Convolutional Layer	13
5.2.2 Pooling Layer	13
5.2.3 Dense Layer	14
5.2.4 Regularization	14
5.2.5 Activation Functions	15
5.2.6 Loss Function	16
5.3 CNN Architectures	16
5.3.1 VGG16	17
5.3.2 VGG19	17
5.3.3 DenseNet121	18
5.3.4 AlexNet	18
5.3.5 EfficientNet-B0	19
5.3.6 InceptionV3	20
5.4 Our Approach	20
5.4.1 Part 1: Assessment of the Level of Freshness	21
5.4.2 Part 2: Assessment of the Level of Healthiness	21

6	Experimental Setup	21
6.1	Training and Testing	21
6.2	Framework	22
6.3	Deployment	22
6.4	Performance Metrics	22
7	Experiments & Discussion	23
7.1	Part 1: VGG16 on the FruitVeg database	23
7.2	Part 2: Deep Learning models on the Tomatoes Subset of the PlantVillage database	25
8	Results	26
8.1	Part 1: VGG16 for FruitVeg database	26
8.2	Part 2: CNNs for Tomatoes/PlantVillage database	29
9	Conclusion & Future work	32
	References	33

LIST OF FIGURES

1	Percentage of food waste by category [3]	1
2	Rejected tomatoes being dumped in a Bundaberg field [4]	3
3	General project workflow plan	7
4	Some Instances of the FruitVeg Dataset 4.1	10
5	Some Instances of the PlantVillage Dataset (a subset of tomatoes) 4.2	11
6	Simple neural network [5]	12
7	Example of a Filter Applied to a Two-Dimensional Input to Cre- ate a Feature Map [6]	14
8	Diagram of Dropout Within a Neural Network [7]	15
9	VGG16 Architecture [8]	17
10	EfficientNet-B0 architecture [9]	19
11	An Example of Gradio GUI to Predict the Disease of the Tomato	22
12	Training Accuracy and Loss for VGG16	26
13	Confusion Matrix for VGG16	27
14	Training Accuracy and Loss of EfficientNet-B0	30
15	Confusion Matrix for EfficientNet-B0 (best model)	31

LIST OF TABLES

1	FruitVeg: Count of Each Fruit and Vegetable In 3 Different Categories	9
2	PlantVillage: Class-Wise Image Distribution of the Tomatoes . .	10
3	Experimental Results	24
4	Classification Report of VGG16	28
5	Performance of Different Models for PlantVillage Dataset	29
6	Final Accuracies of Different Architectures on the PlantVillage Dataset	29
7	Classification Report of EfficientNet-B0	30

LIST OF ABBREVIATIONS

ANN	Artificial Neural Network
CNN	Convolutional Neural Network
VGG16	Visual Geometry Group 16
YOLO	You Only Look Once
VGG19	Visual Geometry Group 19
DenseNet	Dense Convolutional Network
ResNet	Residual Network
Xception	Extreme Inception
RGB	red, green, and blue
ReLU	Rectified linear unit
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
CE	Cross Entropy
AdaM	Adaptive Moment

1 Introduction

Nowadays, food waste stands as a formidable problem with far-reaching effects on social health and environmental sustainability. Many edible products are thrown away each year, due to consumer behavior and inefficiencies in the food supply chain. Approximately one-third of all food produced for human consumption (~ 2.5 billion tons) is lost or wasted globally each year[10, 11]. As shown in Figure 1, fruits and vegetables are the top edible foods that are most likely to be thrown away with a percentage of 39%. To reduce such wastage, automated cutting-edge technologies become a must. Our project addresses this critical concern by aiming to implement an advanced deep-learning model capable of identifying fruits and vegetables and categorizing their level of freshness and healthiness. This system can be utilized in retail establishments and smart fridges, providing consumers with real-time information about the level of freshness and healthiness of the food. The system can also be integrated into distribution centers and warehouses, reducing the possibility that perishable goods would expire before being purchased by customers.

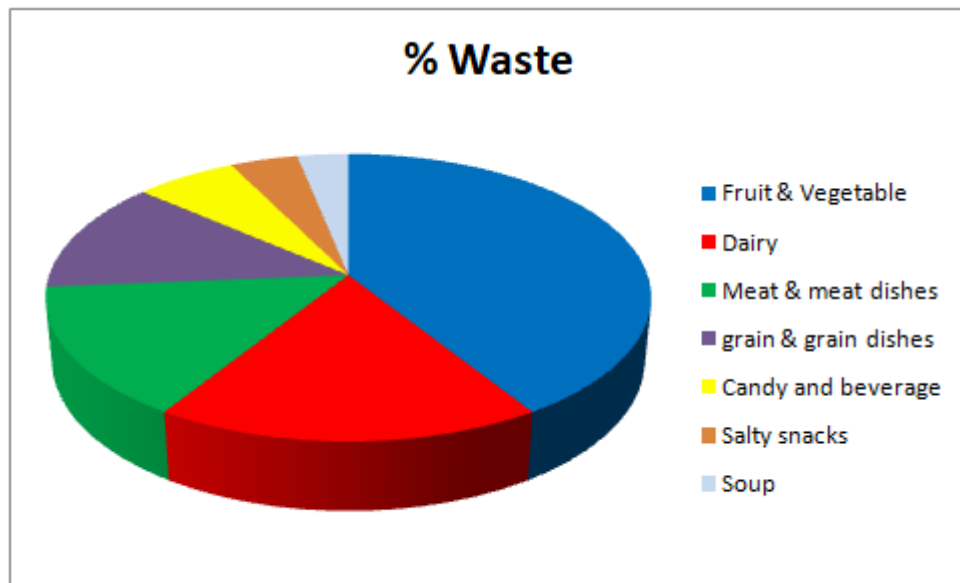


Figure 1: Percentage of food waste by category [3]

Although there are numerous methods to implement a classification model for the identification of fruits and vegetables, the number of classes observed plays an important role. Existing solutions categorize the freshness levels as fresh and rotten only, making it a binary classification problem. Another approach categorizes freshness into three levels fresh, medium, and rotten, introducing a multi-class problem for a more sophisticated and practical dimension to freshness assessment. Our project builds upon the work presented in the solution [1] proposed. The study utilizes two models: VGG16 and YOLOv5. VGG16 is primarily used for classification, and YOLOv5 is used for classification and localization of the object in the image. Our objective is to challenge its efficacy and accuracy. Through different experimentation and fine-tuning, we seek not merely to replicate but to exceed the accuracies delineated by the benchmark paper. Furthermore, we also aim to implement different architectures and different datasets to reach the maximal performance.

Tomatoes are one of the most widely cultivated and consumed vegetables globally; it is the second most important fruit or vegetable crop after potato [12]. Tomatoes represent a significant part of the agricultural economy, with millions of tons produced annually, making them a significant contributor to overall fruit and vegetable waste. The spread of tomato diseases poses major challenges to agricultural sustainability and food security. Diseases such as blight, mold, and bacterial infections, can lead to substantial crop damage if not immediately identified and addressed. Tomato diseases can also cause a substantial loss of yield, leading to a higher incidence of unusable tomatoes that are discarded. This contributes to food waste, as a significant portion of the crop never reaches consumers [13]. Almost 90% of edible tomatoes are thrown away based on appearance alone. Strict aesthetic standards enforced by supermarkets and consumer purchasing behavior lead to a high rate of food loss and waste, as tomatoes that do not meet these standards are often discarded [4] as shown in Figure 2 where rejected tomatoes are being dumped. This research challenges a subset of the PlantVillage database [14] related to tomatoes and uses a computer vision approach to classify and identify various tomato diseases, through utilizing 6 CNN models, DenseNet121, VGG19, MobileNet, and many more models. A CNN model for tomato disease classification can significantly contribute to reducing food waste by enhancing disease management, optimizing resource use, improving supply chain decisions, and increasing consumer confidence in the quality of tomatoes.



Figure 2: Rejected tomatoes being dumped in a Bundaberg field [4]

The rest of the work is organized as follows. Section 2, the Related Works, overviews the previous work and existing solutions done for the problem. Section 3, outlines the project design process. Section 4 summarizes the dataset used for this project. Section 5, the Methodology, contains the necessary information on neural networks, the model architectures, and our approach. Section 6 discusses the experimental setup of the project. Section 7 details the experiments done and discussion. Section 8, shows our work’s results. Finally, Section 9, concludes this project and discusses future work.

2 Related Works

This section is divided into two main parts, firstly, we list past papers challenging the ”freshness” issue and then the previous works on ”healthiness”. In both cases, we have chosen one benchmark paper, and we replicated its experiments.

When examining the relevant publications and literature in the field of

freshness of fruits and vegetables, it is apparent that most studies focus on a binary class classification problem with either fruits or vegetables. Most studies focus on either category, with some focusing on a specific fruit or vegetable. However, the study chosen as the benchmark paper focuses on both fruits and vegetables with a three-class classification: fresh, medium-fresh, and rotten. This is a lot more interesting as it is more descriptive of the actual state of the fruit/vegetable, and covers a more broad and applicable field of study. According to [1], they gather a dataset comprising 60,000 images of 11 different fruits and vegetables, divide them into three classes each (fresh, medium-fresh, and rotten), then use VGG16 and YOLOv5 architectures for image recognition and categorization. They achieved an accuracy of 82% using only VGG16, and 84% when combining VGG16 and YOLOv5 architectures. Within the VGG16 model, they use the Softmax activation function in the model's output layer due to the problem's multi-class nature (fresh, medium-fresh, and rotten). Additionally, they use the Categorical Cross Entropy Loss Function due to multi-class classification. Regarding YOLO, they use the Leaky ReLU activation function in hidden layers along with the sigmoid activation function in the final output layer. Furthermore, in [15], they utilized several CNN architectures and compared the results. The architectures examined are ResNet50, DenseNet, MobileNetV2, InceptionV3, and VGG16. The accuracies achieved were 90.8%, 87.3%, 88.5%, 87.3%, and 80.4% respectively. They proposed a binary classification of fruits with a relatively small dataset of 524 images.

In [16], they primarily utilized the AlexNet architecture on a dataset of 10,901 images of 3 types of fruits. The AlexNet architecture they used consisted of five convolutional layers and three fully-connected layers. Furthermore, they use transfer learning and fine-tune the CNN. They also use the softmax classifier for final classification. They classified the images in a binary classification. They were able to achieve an accuracy of 99%. Next, in [17], they implemented 5 different CNN architectures and compared the results. They proposed a binary classification of 5 types of fruits with a dataset of 5,658 images. The architectures tested were InceptionV3, Xception, VGG16, MobileNet, and NASNetMobile. The accuracies achieved were 97.34%, 97.16%, 96.54%, 95.47%, and 75.29% respectively.

In [18], the authors did not utilize a pre-defined architecture as most others did but utilized a regular Sequential CNN. They used three convolutional layers, with the filters increasing incrementally from 32 to 64 to 128 on each

layer. They also used max-pooling layers with a kernel size of 2x2. Furthermore, they utilized dropout with a dropping rate of 0.25 to avoid overfitting the model. Additionally, they used the ReLU activation function for the fully connected/dense layer and the SoftMax activation function for the output layer. Their model classifies 3 different fruits into a binary class selection, meaning there are 6 total classes, with a dataset of 13,600 images from Kaggle. Lastly, in [19], utilize ResNet, VGG-11, and GoogLeNet architectures with the AlexNet architecture as the base network and YOLO for extracting the region of interest from digital images to classify the freshness grade for fruits. They examined 6 different fruits with a dataset of size 4,000. They have a unique proposition that aims to classify fruits and vegetables into a grade from 0.0 to 10.0, but this also comes with the problem of an extremely limited dataset. This is why they had to gather their dataset, which came with its limitations. On average, their mean squared error (MSE) average values of training and validation sets for the ResNet architecture are 3.582 and 4.058 respectively.

For what is concerning the level of healthiness of fruits and vegetables, various researchers have explored different models and techniques to enhance accuracy and efficiency for plant disease detection. One notable approach [2], which will be our benchmark paper for the study of the healthiness of fruits and vegetables, involved utilizing a pre-trained DenseNet121 model. The authors employed the PlantVillage dataset and took 16,012 tomato images, a widely-used collection of plant leaf images with various diseases, along with 4,000 synthetic images generated by a Conditional Generative Adversarial Network (C-GAN), in total 20,012 images. The synthetic images generated by C-GAN improved the model’s generalizability. The model was trained for three classification tasks: 5-class, 7-class, and 10-class classifications, which correspond to different types of diseases and healthy leaves. The accuracy performance of the DenseNet121 model for 5-class classification was 98.16% on the original PlantVillage dataset and 99.51% with the augmented dataset, and the 7-class classification was 95.08% on the original dataset, and 98.65% with the augmented dataset, and the 10-class classification were 94.34% on the original dataset, and 97.11% with the augmented dataset.

Next, in [20] they utilized their own CNN model. They used three convolution layers paired with max-pooling layers and used 17,500 images from the PlantVillage dataset. The researchers aimed to classify the images into 10 categories. Along with their own CNN model, they also evaluated three other architectures, VGG16, MobileNet and InceptionV3 for comparison. The accuracies achieved were 91.2%, 77.2%, 63.75%, and 63.4% respectively. Furthermore, in [21] They collected their dataset encompassing of 87,848 images, spanning 25 diverse plant species and 58 unique combinations of plant and disease classes. They utilized AlexNet, AlexNetOWTBn, GoogLeNet, Overfeat, and VGG. The accuracies achieved were 99.06%, 99.44%, 97.27%, 98.96% and 99.53% respectively. The VGG model attains the highest accuracy in accurately identifying the plant disease.

Overall, our model repeats the experiment of Fahad et al. [1] in the sense that we take a three-class approach to each fruit and vegetable, meaning we have 33 classes in total. This approach is unique as most models focus on either fruits or vegetables, but we want to create a more holistic and applicable model, therefore we challenged the 33-class issue proposed by Fahad et al. Furthermore, our research also replicates the experiment of Abbas et al [2] focusing on a 10-class problem representing various tomato leaf diseases. We aim to develop a comprehensive and practical model that addresses a wide range of issues in tomato plant health and to achieve higher accuracy and a better-performing model. Further research and development may be necessary to finalize and fully realize the potential of our approach, however, our initial results are very promising and demonstrate the feasibility of our approach.

3 Project Design

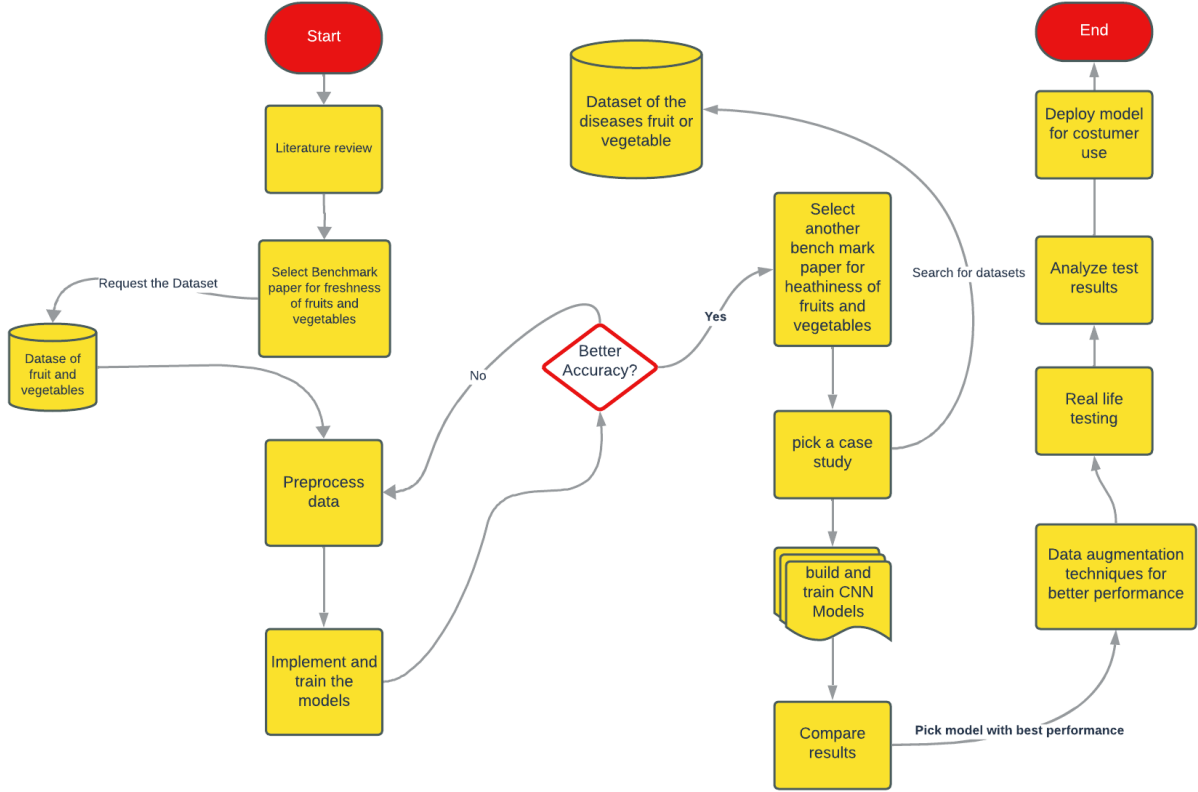


Figure 3: General project workflow plan

Figure 3 shows the general workflow of the project. We start with a literature review and select a benchmark paper for the study of the freshness of fruits and vegetables that serves as a reference for existing methodologies. After preprocessing the data, we train the model proposed in the chosen benchmark paper[1]. If our result is better than theirs, the next step involves picking another benchmark paper for the healthiness of fruits and vegetables and finding a case study for detecting the possible diseases for a specific fruit or vegetable that can lead to wastage. We settled on tomatoes as one of the main contributors to overall fruit and vegetable waste due to the variety of diseases that can affect the health of the vegetable. There are over 19 diseases of tomatoes [22]. These diseases can render tomatoes inedible due to rotting, discoloration, or other quality issues. This leads to a high percentage of the crop being discarded, even if parts of the tomato are still edible. After picking the right dataset, we aim to implement and build different neural

network architectures and compare the results. After identifying the most effective model that yields optimal performance in accurately classifying the disease of the tomato, data augmentation techniques will be added to boost the performance and diversify the dataset as demonstrated in [2]. Finally, we test the model with real-life data and analyze the results. to deploy the model for usage.

4 Dataset

Two datasets were used in this study, the FruitVeg dataset and the PlantVillage dataset. FruitVeg was used to implement the VGG16 model to challenge the paper of Fahad et al [1]. PlantVillage dataset was used to challenge the study of [2] by implementing 6 different models for classifying the diseases of tomatoes. The details of each dataset are shown in the following sections.

4.1 FruitVeg Dataset

The dataset was originally used by [1] in their work. It is publicly available for the research community and can be obtained upon request. It contains a total of 60,059 RGB images of 11 types of fruits and vegetables. six fruits (Apple, Banana, Guava, Orange, Lemon and Tomato) and five vegetables (Brinjal, Cucumber, Chili, Pepper, and Potato) The number of each fruit/vegetable in each category is shown in table 1.

The size of each image is 1800x1800. The images are diverse, containing differing backgrounds and either singular or multiple objects. One point to note is that different fruits and vegetables may appear similar because of their shape or color. For example, a lemon can be confused with an orange, the shape of a cucumber is similar to a green banana, etc. This makes it a challenging data set. The dataset has been split into a training and testing set with a ratio of 70-30, with 70% of the entire dataset for training, 20% of the training subset for validation, and 30% of the entire dataset for testing. The sizes of the subsets are 33,388, 8,332, and 18,336 for training, validation, and testing respectively.

Table 1: FruitVeg: Count of Each Fruit and Vegetable In 3 Different Categories

Name	Fresh	Medium	Rotten
Apple	4082	2975	2785
Banana	2530	899	1999
Orange	2335	1287	1988
Tomato	4261	4715	859
Guava	768	1696	773
Lemon	2281	2881	4033
Brinjal	999	1240	528
Chilies	1083	1392	805
Cucumber	775	857	682
Pepper	2478	927	980
Potato	2115	971	1080
Total	23,707	19,840	16,512

4.2 PlantVillage Dataset

The PlantVillage dataset we chose to utilize for tomato leaf diseases is public [14]. It was created to address the need to increase food production by an estimated 70% by 2050. With infectious diseases causing up to 40% crop losses. The dataset contains a total of 54,305 RGB images of multiple fruits and vegetable leaves diseased, including apples, tomatoes, blueberries, cherries, corn, peaches, potatoes, and raspberries. Specifically for tomato leaf diseases, contains a total of 18,160 RGB images. This subset includes 10 different classes: 9 classes for various diseases and 1 class for healthy tomato leaves. The tomato subset was originally used by [2]. In their work, they only used 16,012 images from the tomato subset, we used 18,160 images. The images come with varying backgrounds, making it a challenge for image classification tasks. The number of each tomato leaf disease is shown in table 2.

We split the dataset into three segments, which are training, validation, and test sets. Based on [23], we split the dataset into 60%, 10%, and 30% respectively, resulting in 10,896 images for training, 1,816 images for validation, and 5,448 images for testing.

Table 2: PlantVillage: Class-Wise Image Distribution of the Tomatoes

Name	Number of images
Tomato Yellow Leaf Curl Virus (YLCV)	5357
Tomato Bacterial Spot (Bctsp)	2127
Tomato Late Blight (TLB)	1909
Tomato Septoria leaf spot (SptL)	1771
Tomato Two Spotted Spider Mite (SpdM)	1676
Tomato Target Spot (TTS)	1404
Tomato Early Blight (TEB)	1000
Tomato Leaf Mold (LMld)	952
Tomato Mosaic Virus (MscV)	373
Tomato Healthy (Hlth)	1591
Total	18,160

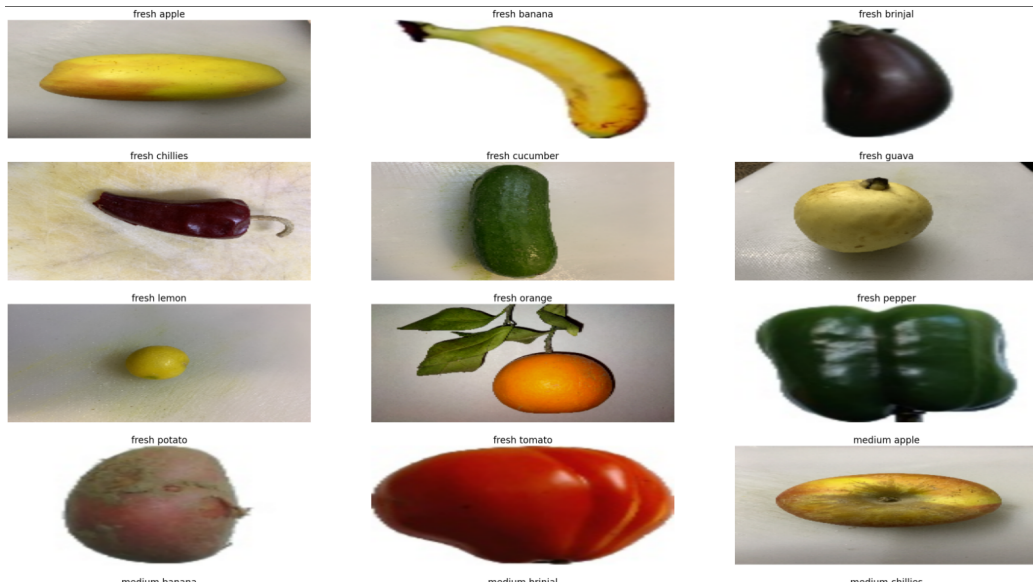


Figure 4: Some Instances of the FruitVeg Dataset 4.1

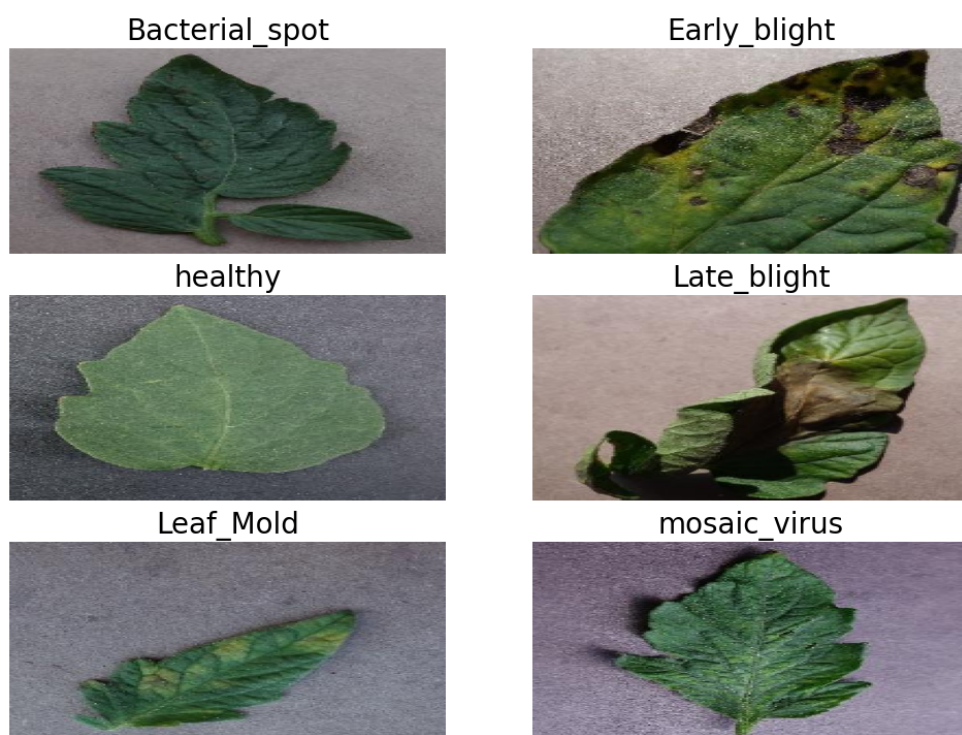


Figure 5: Some Instances of the PlantVillage Dataset (a subset of tomatoes)
4.2

5 Methodology

5.1 Neural Networks

Neural networks, also known as artificial neural networks (ANNs) are a subset of machine learning and are at the heart of deep learning algorithms [24]. The name and structure of neural networks are inspired by the human brain and closely mimic the way that biological neurons signal to one another. ANNs contain an input layer, one or more hidden layers, and an output layer, as shown in figure 6.

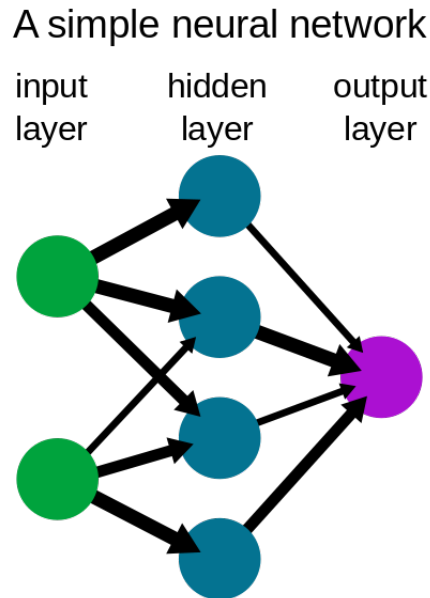


Figure 6: Simple neural network [5]

Each node within an ANN connects to another and has an associated weight and threshold. If the output of any individual node exceeds a specified threshold value, the node is activated and the data contained within the node is transferred to the next layer of the network. Otherwise, if the threshold value is not exceeded, no data is transferred to the next layer of the network [24].

5.2 Convolutional Neural Networks (CNN)

A Convolutional Neural Network (CNN) is a specialized kind of neural network that is superior at distinguishing unique features in images, speech, or

audio signal inputs [25]. CNNs have three main types of layers: convolutional layers, pooling layers, and fully connected (FC) layers. Fully connected layers are also called dense layers.

The convolutional layer is the first layer in a convolutional network. The convolutional layer can be followed by additional convolutional layers or pooling layers, however, the final layer will always be the fully connected/dense layer. With each additional layer, the CNN increases in complexity. Generally, the first few layers in a CNN focus on extracting simple features such as color and edges, and as the image data progresses through the layers in a CNN, larger elements or shapes of the object start to be recognized which leads to the identification of the intended object [25].

5.2.1 Convolutional Layer

The convolutional layer is a fundamental building block of a CNN, where the majority of the computation occurs [25]. The convolutional layer only requires the input data, a filter, and a feature map. For example, we have a colored image represented as a matrix of pixels in 3D. This means that we will have three dimensions: width, height, and depth of the image which correspond to its RGB values. Additionally, we have the kernel, or filter, which is moved across the receptive fields of the image, checking if the desired feature is present. This process is known as convolution [25]. The result of the convolution is stored in the feature map. This is shown in diagram 7.

5.2.2 Pooling Layer

The pooling layer downsamples the number of parameters in the input and extracts crucial information. Pooling results in a lot of information loss, however, there are several benefits to utilizing it. Pooling layers help CNNs reduce complexity, improve efficiency, and limit the risk of overfitting [25].

$$Y_{ij} = \max_{m,n} (X_{p \cdot i + m, p \cdot j + n}) \quad (1)$$

Equation 1 portrays a max-pooling layer. Y represents the output of the pooling layer at position (i,j) , and $\max$ denotes the maximum operation over the elements of the specified region. X represents the input feature map at position $(p \cdot i + m, p \cdot j + n)$ in the original input.

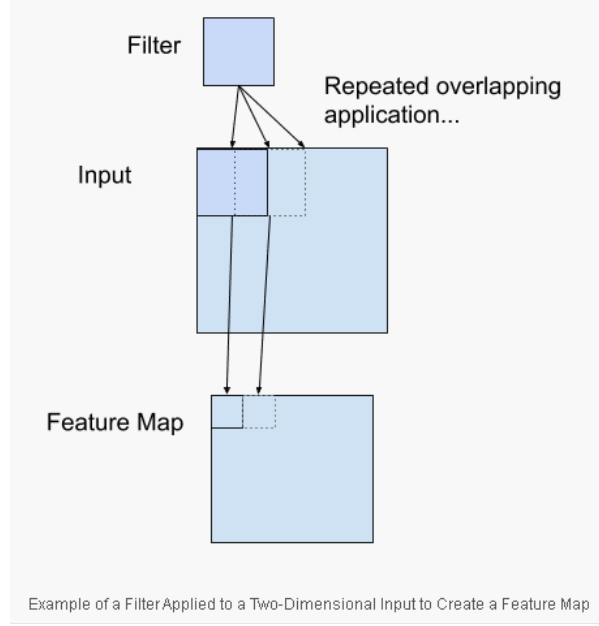


Figure 7: Example of a Filter Applied to a Two-Dimensional Input to Create a Feature Map [6]

5.2.3 Dense Layer

The dense layer, or fully connected layer, in a neural network, connects all nodes in the output layer directly to a node in the previous layer. Each node contains several inputs, weights, a bias term, and an activation function [25].

$$Y = a(X \cdot W + b) \quad (2)$$

Equation 2 displays the dense layer output formula, Where 'a' represents the activation function used in the layer, X is the input vector, W is the weight matrix, b is the bias, and $\cdot$ denotes the dot product.

5.2.4 Regularization

Regularization is a set of methods that serve primarily as a means to reduce overfitting in machine learning models. This is often done with a tradeoff of a marginal decrease in training accuracy for an increase in model generalizability [7].

Dropout is a special regularization technique that is used within this study. Dropout regularizes neural networks by randomly dropping out nodes, along with any input and output connections, from the network during training. This is shown in 8. The nodes dropped during a certain epoch are randomly

chosen. Dropout results in an enhancement in the robustness and generalizability of a model, and ultimately reduces overfitting.

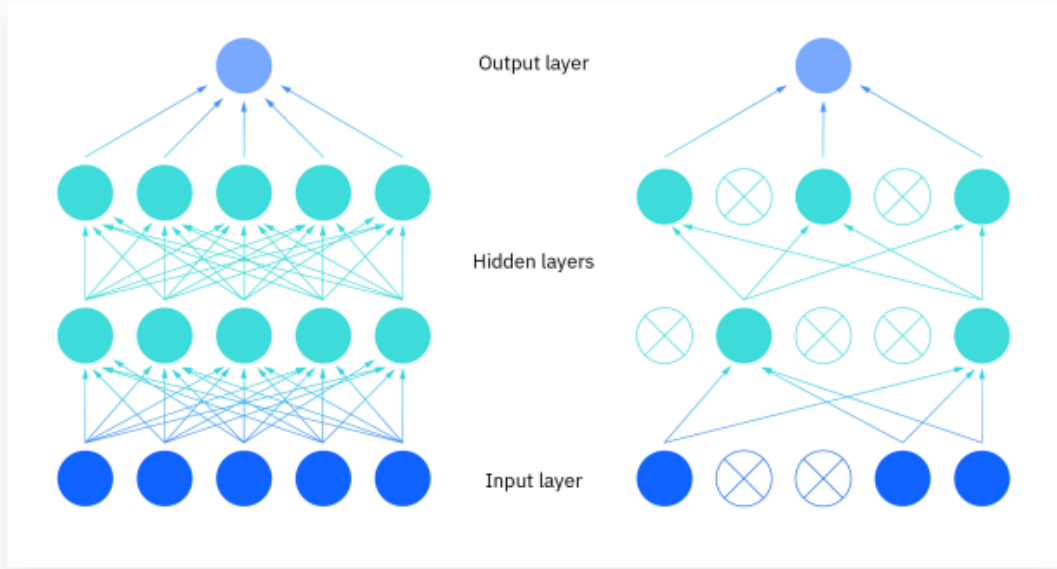


Figure 8: Diagram of Dropout Within a Neural Network [7]

5.2.5 Activation Functions

Activation functions are non-linear which allows for the approximation of complex functions and also provides non-linearities into a network [26]. Since real-life data is often non-linear, and models generally focus on real-life applications, activation functions are crucial for the success of any feasible neural network.

$$\text{ReLU}(x) = \max(0, x) \quad (3)$$

Equation 3 shows the Rectified Linear Unit (ReLU) activation function. The equation states that the ReLU of x is the maximum of 0 and x . If x is positive, the ReLU function returns x ; otherwise, it returns 0. This is a widely used function in real-time implementations. Its popularity generally stems from the computational simplicity of the function, linear behavior, and the capability of outputting a true zero value (representational sparsity) [27]. Due to the aforementioned reasons, we will be utilizing the ReLU activation functions in most of our fully connected/dense layers.

$$s(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \quad (4)$$

Equation 4 shows the softmax activation function, where z is the output vector, and C is the number of classes. The softmax activation function transforms raw outputs from a neural network into a vector of probabilities, essentially a probability distribution over the input classes. In a multi-class classification problem, softmax is essential in the output layer of a neural network since it makes raw data from the neural network readily interpretable [28]. Due to the nature of our model, we will also be using the softmax activation function in the output layer.

5.2.6 Loss Function

The loss function, also known as the cost function, measures the difference, or error, between actual and predicted values at a certain position [29]. This improves a machine learning model's efficacy by providing feedback to the model so it can adjust the weights and biases throughout the network to minimize the error and find a local or global minimum. This process continuously iterates, moving along the direction of the steepest descent until the cost function is close to or at zero, at which point the model will stop learning [29].

$$L_{CE} = - \sum_{i=1}^C y_i \cdot \log(\hat{y}_i) \quad (5)$$

Equation 5 shows the categorical cross entropy formula, where y_i is the true labels ranging from 0 to 1, $\hat{y}_i$ is the model prediction and C is the number of classes. Categorical Cross Entropy, also known as "softmax loss", is a softmax activation plus a Cross-Entropy loss used for multiclass classification [30]. Since this loss function specializes in multiclass classification, we will be using it in our model.

5.3 CNN Architectures

For this study, a total of 6 different architectures have been implemented to create a comparison and to find the best architecture for the datasets. The classification CNN models used are VGG16, VGG19, DenseNet121, AlexNet, EfficientNet-B0, and InceptionV3.

5.3.1 VGG16

Visual Geometry Group 16 (VGG16) is a convolutional neural network architecture that was introduced in the ImageNet Large-Scale Visual Recognition Challenge (ILSVRC) event in 2014 by Simonyan et al, a group in the Department of Engineering Science, University of Oxford. They showcased their model in paper [31], which won 1st and 2nd place for object detection and classification. It was able to classify more than a million images of 1000 different classes with 92.7% accuracy[31]. VGG16 is considered one of the best computer vision models [32].

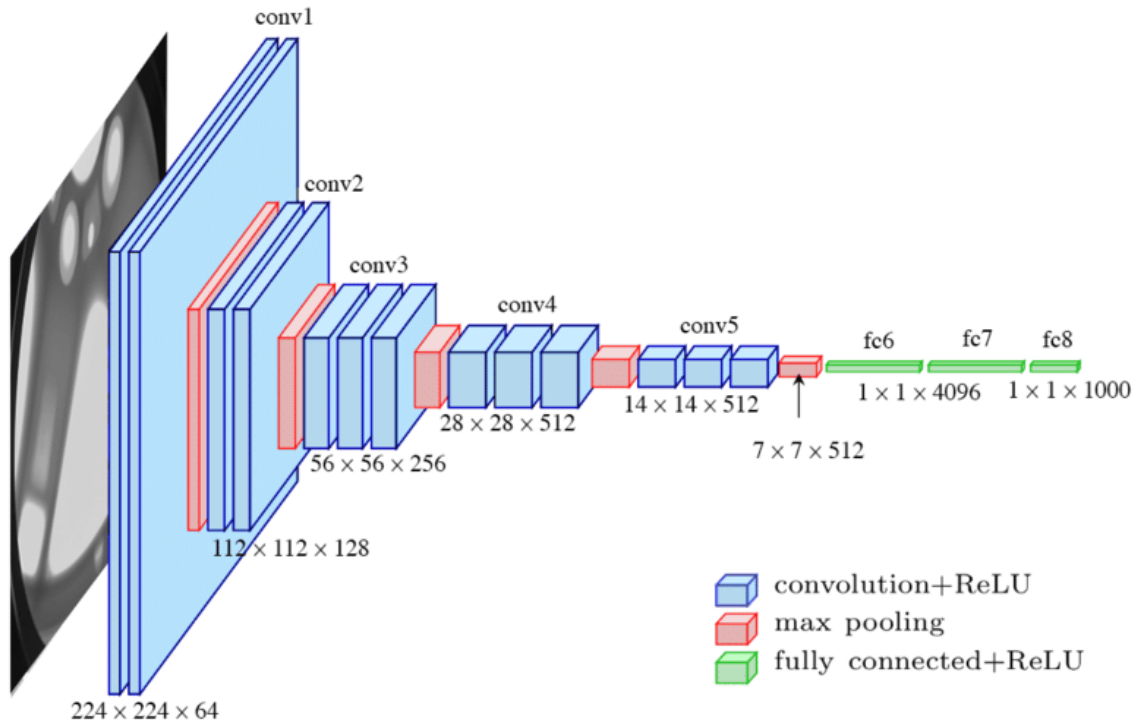


Figure 9: VGG16 Architecture [8]

5.3.2 VGG19

VGG19 also known as VGGNet-19 was introduced by the same group that created VGG16, Visual Geometry Group [31]. VGG19 has the same approach and architecture as VGG16, the only difference is that it supports 19 layers instead of 16 layers. As in VGG16, the number 19 means that there

are 19 learnable layers, which means that VGG19 contains three extra convolutional layers. While VGG16 was already a powerful model, the group wanted to see if adding more layers would lead to better performance. In some cases, VGG19 showed slight improvements over VGG16 in terms of accuracy, although the difference was not significant. It provided evidence that deeper networks could perform better, up to a point.

5.3.3 DenseNet121

DenseNet, a Densely Connected Convolutional Network is a family of CNNs that uses a dense connectivity pattern between layers, which allows for better performance and speed in the network. The architecture was introduced in [33] by Huang et al. It is widely deployed for image classification tasks and has achieved excellent performances on ImageNet, CIFAR-10, and CIFAR-100 datasets. DenseNet is known for its effectiveness on large datasets. The advantage of using DenseNet in classification tasks is that it can achieve better accuracies with fewer learnable parameters. It is also robust to overfitting and can lead to more accurate predictions for its strong feature propagation. DenseNet consists of a series of dense blocks, each block contains convolutional layers. Each block takes the output of the previous block as input, as well as the outputs of all the previous blocks. This creates a dense connectivity pattern between all the layers of the network, which allows the information to flow more efficiently through the network. DenseNet121 and DenseNet201 are two variants of the DenseNet architecture, the difference between the two is that DenseNet121 contains 121 layers while DenseNet201 contains 201 layers, DenseNet201 has almost twice the number of parameters as DenseNet121 which makes it more complex and requires more computational power and memory[34].

5.3.4 AlexNet

AlexNet architecture was introduced by Alex et al [35]. It is a deep CNN that was able to classify the ImageNet dataset which consists of 1.2 million images into 1000 different classes. They achieved top-5 in the ImageNet ILSVRC-2010 contest. AlexNet was the first architecture that used GPU to boost training performance. It consists of 5 convolutional layers, 3 max-pooling layers, 2 normalized layers, 2 dense layers, and 1 softmax layer with a total of 60 million learnable parameters. The input size is 227x227x3. AlexNet is often credited for making the ReLU activation function popular due to the success of AlexNet in the ILSVRC contest which brought attention to ReLU because Old models used the Tanh activation function in their network. This

helped it train faster and achieve better performance compared to networks using tanh or sigmoid functions. AlexNet also introduced the use of dropout regularization technique to reduce overfitting in its fully connected layers [36].

5.3.5 EfficientNet-B0

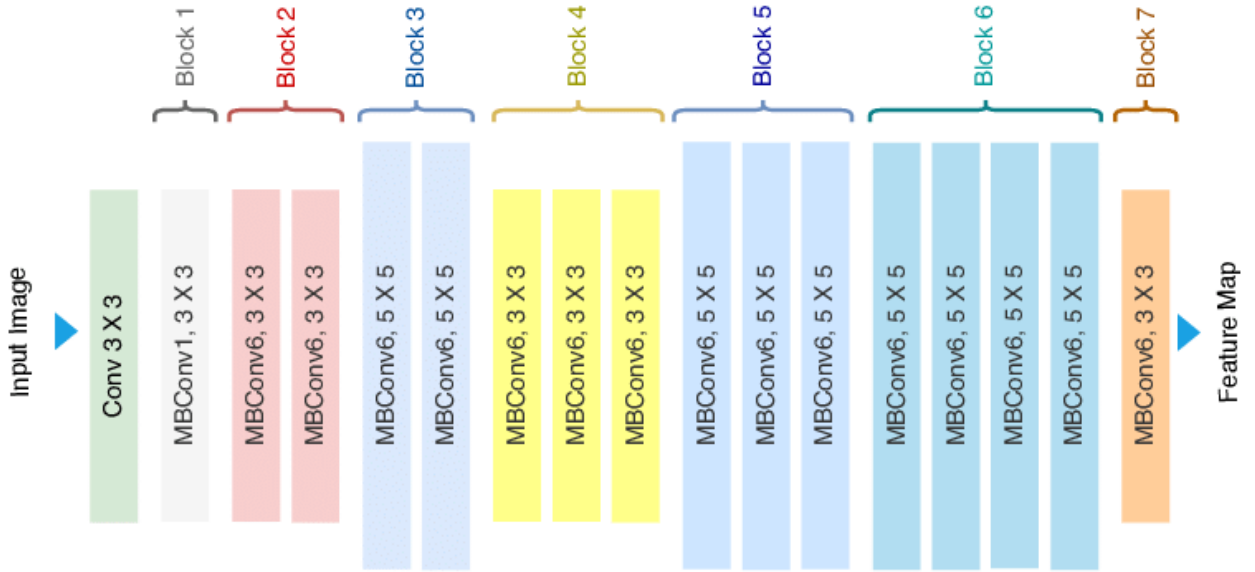


Figure 10: EfficientNet-B0 architecture [9]

EfficientNet is a family of models introduced in 2019 by Mingxing et al [37], a team of researchers at Google AI. Their model achieved top-1 accuracy on ImageNet for being 8 times smaller and 6 times faster than the other best models, it also achieved an accuracy of 92% on CIFAR-100 datasets and many other benchmark datasets.

EfficientNet was built upon the concept of compound scaling which addresses the trade-off between the model's size, accuracy, and computational efficiency. The goal is to scale the three dimensions of the neural network: width, depth, and resolution which can be capable of balancing these dimensions; This is done by performing a systematic grid search to find the optimal combination of width, depth, and resolution.

The EfficientNet-B0 model is part of the EfficientNet family of models, designed for efficient and effective feature extraction. As shown in figure 10, This architecture utilizes depthwise separable convolutions (MBConv), which streamline the computational process by separating the filtering and

combining steps, therefore reducing complexity. Batch normalization is used to stabilize the learning process, and the Swish activation function introduces the non-linearity. Additionally, the Squeeze and Excitation optimization refines the network’s attention to informative features by dynamically adjusting channel-wise feature responses.

EfficientNet-B0 is structured to be compact but powerful, containing 237 layers that work to process and extract features efficiently. The model’s innovative design allows it to achieve high levels of accuracy with significantly reduced computational resources. As the foundational model of the EfficientNet family, B0 sets the stage for its successors, which further expand on its efficiency and accuracy[38].

5.3.6 InceptionV3

Inception, also known as GoogLeNet, was introduced by Google [39]. It achieved top results in ILSVRC 2014. It used 12 times fewer parameters than AlexNet which won the 2012 contest. The basic module of InceptionV1 consists of four parallel layers: 1x1 convolution, 3x3 convolution, 5x5 convolution, and 3x3 max pooling with a total of 22 layers. InceptionV3 is an improved version of inception architecture. It introduces several key improvements, such as factorized convolutions that break down larger convolutions into smaller, more manageable operations, and batch normalization for faster training. The model also employs an auxiliary classifier to distribute gradient information deeper into the network, aiding in the training process. These enhancements enable InceptionV3 to achieve high accuracy while being computationally efficient.

The architecture of InceptionV3 is characterized by its ‘Inception modules,’ which are small networks within the larger network, allowing it to learn at various scales and complexities. Each module contains parallel paths with different types of filters and pooling operations, which are then concatenated. This multi-path design enables the model to adapt to various features in the input data, making it robust and versatile for different image recognition tasks. The model’s success is evidenced by its performance in the ImageNet competition, where it significantly improved upon previous benchmarks.

5.4 Our Approach

Our study is divided into two parts, the first part is to identify the freshness of fruits and vegetables by utilizing VGG16 for the FruitVeg dataset proposed in [1] and try to beat their accuracy. The second part is to identify the healthiness of tomatoes by classifying their diseases. This is done by utilizing

6 different models explained in Section 5.3 for the subset of tomatoes in the PlantVillage dataset and comparing the results to get the best-performing model.

5.4.1 Part 1: Assessment of the Level of Freshness

The FruitVeg database 4.1 has been challenged using the VGG architecture. A predefined VGG16 architecture exists from the Keras library that already implements all of the layers until the last max pooling layer. We merged the predefined architecture and froze its learnable layers. We then added two dense layers with ReLU [3] as the activation function and Softmax [4] as the output layer. A dropout of a rate of 20% was used for regularization. The loss function used for calculating error is the categorical cross-entropy function shown in formula 5 because the model has multiple classes as an output. AdaM optimization was used as our gradient descent method with a learning rate of 0.0001. The model was trained for 15 epochs.

5.4.2 Part 2: Assessment of the Level of Healthiness

For the PlantVillage dataset, 6 different models were implemented using TensorFlow. VGG16, VGG19, InceptionV3, EfficientNet-B0, and DenseNet121 architectures were imported from the Keras library as pre-trained models, and AlexNet was manually implemented. After merging the predefined architecture and freezing its learnable layers, two dense layers were added, the first dense layer has 256 neurons with ReLU as the activation function, and the output layer with softmax. A Batch Normalization layer was added to avoid overfitting and to accelerate the training process followed by a dropout of rate 20% for regularization. Categorical cross entropy was used as the loss function and AdaM was used as our gradient descent method with a learning rate of 0.0001. The models were trained for 20 epochs except for the InceptionV3 model as it was trained for only 5 epochs to avoid overfitting.

6 Experimental Setup

6.1 Training and Testing

Python was used as the programming language for training and testing the models due to its simplicity and readability. Python also boasts a rich ecosystem of libraries such as NumPy, Pandas, OpenCV, SKLearn, and Matplotlib, which are invaluable for data manipulation and visualization.

Python also offers Great frameworks specifically designed for AI and deep learning, such as TensorFlow, Keras, and PyTorch.

6.2 Framework

The team chose Tensorflow due to its comprehensive, flexible ecosystem of tools, libraries, and community resources that enable the construction and deployment of complex deep-learning models. TensorFlow can process large volumes of data with high efficiency and its support for deep learning algorithms made it particularly suitable for the task at hand. Its automatic differentiation capabilities and robust GPU acceleration are instrumental in handling complex neural network architectures.

6.3 Deployment

for model deployment, the Gradio interface is used for many reasons. Firstly, it simplifies the process of creating web-based interfaces for deep learning models. It's designed to be user-friendly, allowing you to deploy a model with just a few lines of code. With Gradio, we can build interactive web applications that allow users to engage with our model directly, providing inputs and seeing outputs in real-time as shown in Figure 11.

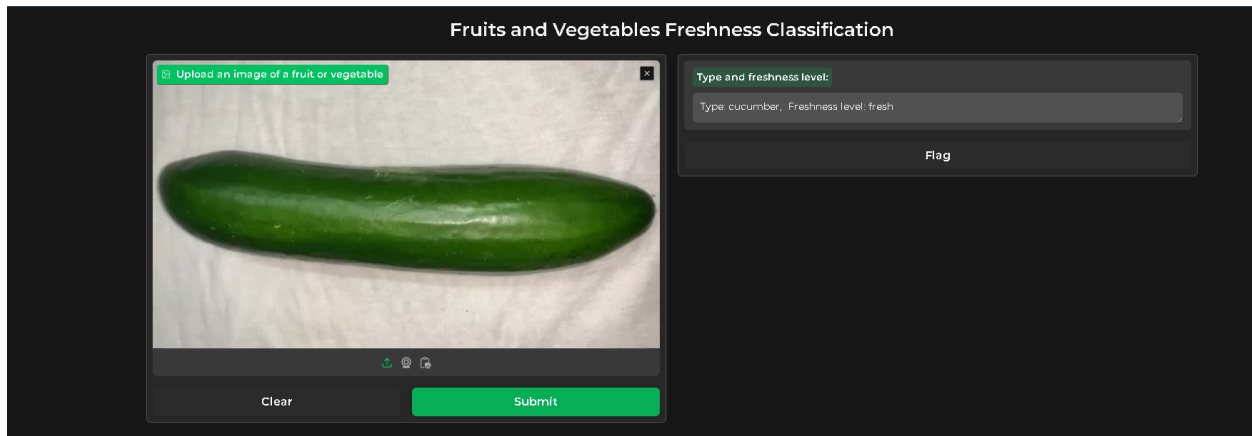


Figure 11: An Example of Gradio GUI to Predict the Disease of the Tomato

6.4 Performance Metrics

To evaluate the performance of the classification models in both experiments, two fundamental metrics are used, the confusion matrix and the classification report. The confusion matrix provides a comprehensive summary of

a model’s predictions and the classification report provides a more detailed overview of the model’s performance. These metrics help us check the model’s ability to identify both positive and negative cases and assess the presence of misclassification cases. The formulas for each metric are shown below.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (6)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (7)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (8)$$

$$\text{F1} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} \quad (9)$$

$$\text{Loss} = \frac{FP + FN}{TP + TN + FP + FN} \quad (10)$$

- True Positive (TP): correctly predicted positive instance
- False Positive (FP): incorrectly predicted positive instance
- True Negative (TN): correctly predicted negative instance
- False Negative (FN): incorrectly predicted negative instance

The formula shown in 6 calculates the accuracy by dividing true predictions by all predictions. In contrast, equation 10 calculates the error or the misclassification by dividing the incorrect predictions by all predictions. Equation 7 shows the precision, which is the capability to detect ONLY present instances, while the Recall shown in 8 detects ALL present instances. Equation 9 calculates the F1 score, which is what percent of positive predictions were correct. In addition to these metrics, Support was also calculated, which is the number of actual occurrences of the class in the data set. Imbalanced support in the training data may indicate structural weaknesses in the reported scores of the model and could indicate the need for rebalancing. Support doesn’t change between models but instead diagnoses the evaluation process[40].

7 Experiments & Discussion

7.1 Part 1: VGG16 on the FruitVeg database

There were four different experiments as shown in Table 3, each varying in terms of epochs, the number of hidden layers, and the number of neurons in

each layer. Experiment 1, with 15 epochs and 2 hidden layers (32 and 16 neurons), showed moderate performance with a training accuracy of 75.84% and a test accuracy of 91.28%. Experiment 2 increased the complexity with 64 and 32 neurons in its layers, resulting in a significant improvement in both training and test accuracy, along with a high F1 Macro score of 0.9102.

In Experiment 3 we increased the model’s capacity with 128 and 64 neurons, reaching the highest training accuracy of 98.32% and an impressive test accuracy of 94.58%. The F1 Macro score also peaked at 0.9207 indicating a balanced performance across different classes. Finally Experiment 4, the most complex model with three layers of 128, 64, and 32 neurons and 20 epochs, slightly trailed behind Experiment 3 in terms of training accuracy and test accuracy, with also a lower F1 Macro score.

Based on these results, experiment 3 appears to be the most effective, achieving the highest test accuracy and F1 Macro score, which are important indicators of a model’s ability to generalize well on new data. The increase in neurons and the maintenance of the number of epochs in Experiment 3 seems to offer the best balance between complexity and performance, which makes it the standout choice among the conducted experiments.

Table 3: Experimental Results

Experiment	Parameters			Metrics			
	epochs	No. of Dense Layers	No. of Neurons in the dense layers	Training Acc	F1 Macro	Test Acc	Loss
1	15	2	32,16	0.7584	0.8539	0.9128	0.3426
2	15	2	64,32	0.9169	0.9102	0.9296	0.3870
3	15	2	128,64	0.9832	0.9207	0.9458	0.2140
4	20	3	128,64,32	0.9675	0.9148	0.9390	0.2493

7.2 Part 2: Deep Learning models on the Tomatoes Subset of the PlantVillage database

During the experimental phase, six different CNN models were trained over three weeks. Each model underwent extensive training and evaluation to determine their performance on the PlantVillage sub-dataset. The training time for each model varied depending on the complexity of the architecture, but on average, each model required approximately 5-10 hours of continuous training to complete 20 epochs.

To ensure fair comparison and consistency, all models were implemented with the same structure as shown in Code 1, consisting of a single dense layer with 256 neurons followed by a batch normalization layer and an output layer with 10 classes. The models were trained using the Adam optimizer with a learning rate of 0.0001, which provided a balance between speed and accuracy during backpropagation. The categorical cross-entropy loss function explained in Section 5.2.6 was used to measure the error rate, making it suitable for multi-class classification problems.

```
1 model= Sequential()  
2 model.add(arch) # add the predefined architecture  
3 model.add(Flatten())  
4 model.add(Dense(256, activation='relu')) # relu as activation  
   function  
5 model.add(layers.BatchNormalization()) #for faster training  
6 model.add(Dropout(0.2)) # for regularization and avoiding  
   overfitting  
7 model.add(Dense(10, activation='softmax', name='Disease')) #  
   outputs 10 classes  
8  
9 #compile  
10 model.compile(optimizer=Adam(learning_rate=0.0001),  
11               loss='categorical_crossentropy', # the loss  
               function used for finding error ,  
12               metrics=['acc'])  
13  
14 csv_logger = CSVLogger('training.log', separator=',', append=  
   False) # to save the model metrics in a log file  
15  
16 history = model.fit(train_generator, validation_data=  
   validation_generator, epochs=20, callbacks=[csv_logger])
```

Code 1: Algorithm Used For Training

Several experiments were conducted to fine-tune the models, including adjusting the dropout rate, experimenting with different batch sizes, and adding more hidden layers. These experiments aimed to enhance the gener-

alization capability of the models and reduce overfitting. The results from these experiments were logged using the CSVLogger callback (line 14), for a detailed analysis and visualization of training metrics such as accuracy and loss over each epoch.

8 Results

8.1 Part 1: VGG16 for FruitVeg database

After picking the best performance model from the experiments shown in Section 7 as well as Figure 14, we achieved a training accuracy of 98.32%, validation accuracy of 89.85%, and a loss of 0.05% and validation loss of 0.36% with no sign of overfitting. For evaluation, the confusion matrix and the classification report were computed as shown in Figure 13 and Table 3. We achieved an average precision of 0.93, an average recall of 0.92, an average F1 score of 0.92, and an overall accuracy of 94.58%.

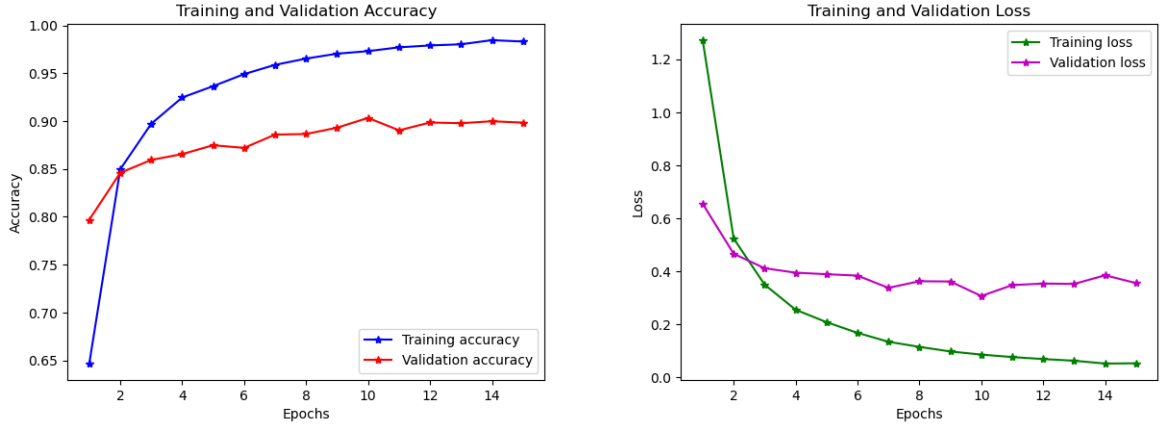


Figure 12: Training Accuracy and Loss for VGG16

The confusion matrix is shown in Figure 13. We can see that the best performance model (Experiment 3 shown in Table 3) generally performs well on certain classes, such as fresh banana, fresh lemon, medium tomato, and rotten potato, where the number of correct predictions is high (noted by the high numbers along the diagonal). However, there are a total of 994 misclassifications with a loss of 0.214. For instance, fresh apples are often misclassified as medium apples and fresh tomatoes as medium tomatoes. These errors could be due to similarities in features between fresh and medium states of the same item which can also be confusing for humans. What one person sees as a fresh apple might be seen as a medium fresh apple by another, depending

on personal standards or experiences. Factors such as color, texture changes, and minor blemishes can make the classification subjective.

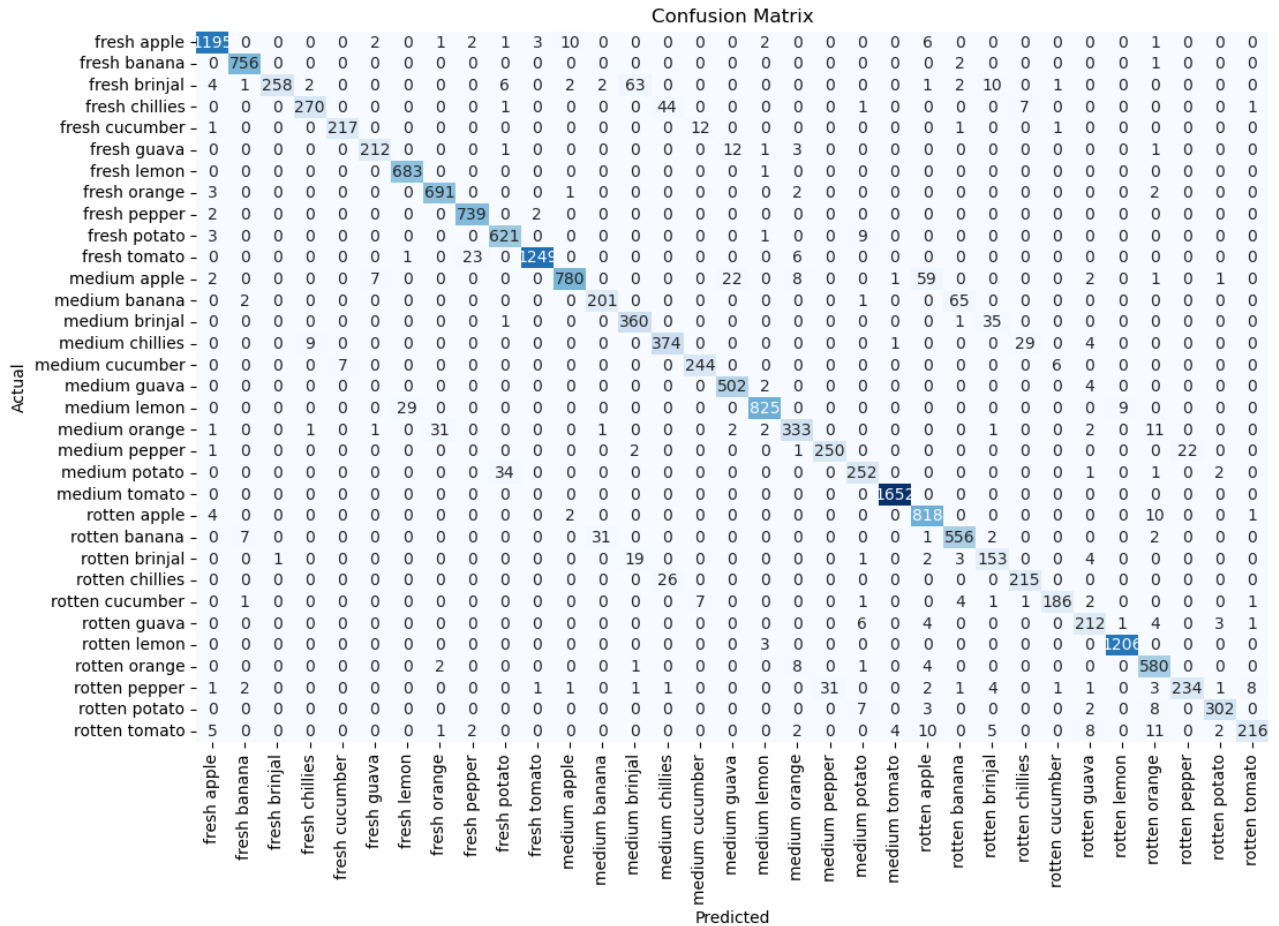


Figure 13: Confusion Matrix for VGG16

Table 4: Classification Report of VGG16

Class	Precision	Recall	F1-Score	Support
fresh apple	0.9779	0.9771	0.9775	1223
fresh banana	0.9831	0.9960	0.9895	759
fresh brinjal	0.9961	0.7330	0.8445	352
fresh chillies	0.9574	0.8333	0.8911	324
fresh cucumber	0.9688	0.9353	0.9518	232
fresh guava	0.9550	0.9217	0.9381	230
fresh lemon	0.9579	0.9985	0.9778	684
fresh orange	0.9518	0.9886	0.9698	699
fresh pepper	0.9648	0.9946	0.9795	743
fresh potato	0.9338	0.9795	0.9561	634
fresh tomato	0.9952	0.9765	0.9858	1279
medium apple	0.9799	0.8834	0.9291	883
medium banana	0.8553	0.7472	0.7976	269
medium brinjal	0.8072	0.9068	0.8541	397
medium chillies	0.8404	0.8969	0.8677	417
medium cucumber	0.9278	0.9494	0.9385	257
medium guava	0.9331	0.9882	0.9598	508
medium lemon	0.9857	0.9560	0.9706	863
medium orange	0.9174	0.8627	0.8892	386
medium pepper	0.8897	0.9058	0.8977	276
medium potato	0.9032	0.8690	0.8858	290
medium tomato	0.9964	1.0000	0.9982	1652
rotten apple	0.8989	0.9796	0.9375	835
rotten banana	0.8756	0.9282	0.9011	599
rotten brinjal	0.7251	0.8361	0.7766	183
rotten chillies	0.8532	0.8921	0.8722	241
rotten cucumber	0.9538	0.9118	0.9323	204
rotten guava	0.8760	0.9177	0.8964	231
rotten lemon	0.9918	0.9975	0.9946	1209
rotten orange	0.9119	0.9732	0.9416	596
rotten pepper	0.9141	0.7986	0.8525	293
rotten potato	0.9711	0.9379	0.9542	322
rotten tomato	0.9474	0.8120	0.8745	266
Accuracy			0.9458	18336
Macro Avg	0.9272	0.9177	0.9207	18336
Weighted Avg	0.9474	0.9458	0.9454	18336

8.2 Part 2: CNNs for Tomatoes/PlantVillage database

Table 5: Performance of Different Models for PlantVillage Dataset

Model	Training Accuracy	Test Accuracy	Precision	Recall	F1-score	Loss
VGG16	0.9231	0.9079	0.8862	0.8811	0.8830	0.2373
VGG19	0.9081	0.8766	0.8539	0.8410	0.8419	0.2753
DenseNet121	0.9804	0.9487	0.9361	0.9305	0.9325	0.0723
AlexNet	0.9895	0.9471	0.9378	0.9293	0.9329	0.0336
InceptionV3	0.9990	0.9287	0.9202	0.8969	0.9059	0.0142
EfficientNet-B0	0.9726	0.9604	0.9485	0.9502	0.9493	0.0906

The performance metrics of the implemented models are shown in table 5 with **EfficientNet-B0** as the best model with an accuracy of **96.04%**. InceptionV3 has the highest training accuracy due to its complexity but wasn't good for the regularization of unseen data. The accuracy of Abbas et al [2] for the DenseNet121 model without the C-GAN augmentation was 94.34%, which we beat using our model by **1.7** percent points. Future work will include working on the C-GAN augmentation technique along with other techniques to ensure the best performance and generalization. The final accuracies of our implemented models are shown in 6.

Table 6: Final Accuracies of Different Architectures on the PlantVillage Dataset

Architecture	Accuracy
VGG16	90.79%
VGG19	87.66%
DenseNet121	94.87%
InceptionV3	92.87%
AlexNet	94.71%
EfficientNet-B0	96.04%

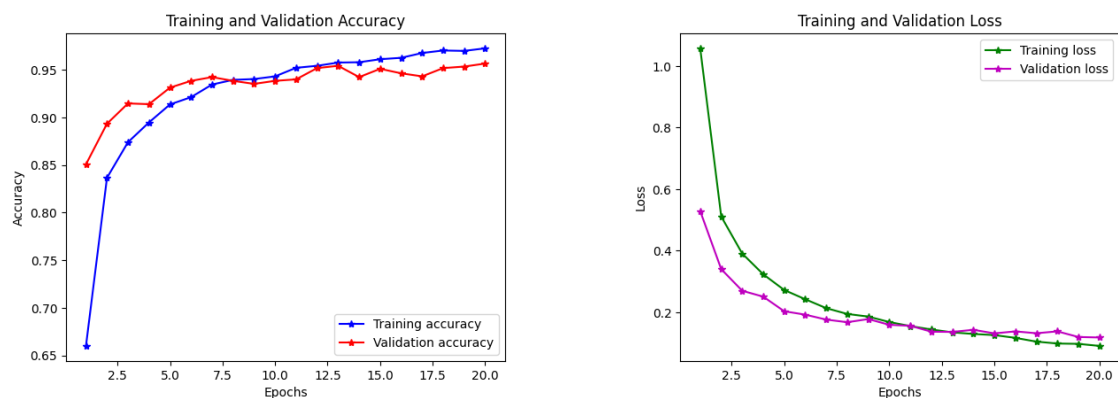


Figure 14: Training Accuracy and Loss of EfficientNet-B0

Table 7: Classification Report of EfficientNet-B0

Class	Precision	Recall	F1-score	Support
Bacterial_spot	0.9749	0.9734	0.9742	639
Early_blight	0.8792	0.8733	0.8763	300
Late_blight	0.9406	0.9389	0.9397	573
Leaf_Mold	0.9472	0.9406	0.9439	286
Septoria_leaf_spot	0.9620	0.9511	0.9565	532
Spider_mites	0.9448	0.9523	0.9485	503
Two-spotted_spider_mite				
Target_Spot	0.8853	0.9147	0.8998	422
Yellow_Leaf_Curl_Virus	0.9944	0.9919	0.9932	1608
healthy	0.9916	0.9833	0.9874	478
mosaic_virus	0.9649	0.9821	0.9735	112
Accuracy			0.9604	5453
Macro avg	0.9485	0.9502	0.9493	5453
Weighted avg	0.9606	0.9604	0.9605	5453

		Confusion Matrix									
Actual	Bacterial_spot -	622	2	6	1	1	0	3	4	0	0
	Early_blight -	3	262	17	2	6	3	5	2	0	0
	Late_blight -	2	14	538	8	2	2	5	0	2	0
	Leaf_Mold -	2	2	5	269	2	2	2	1	0	1
	Septoria_leaf_spot -	4	10	1	1	506	1	8	0	0	1
	Spider_mites Two-spotted_spider_mite -	0	0	1	0	2	479	19	2	0	0
	Target_Spot -	2	6	2	0	5	18	386	0	2	1
	Yellow_Leaf_Curl_Virus -	3	1	2	3	0	2	2	1595	0	0
	healthy -	0	1	0	0	0	0	6	0	470	1
	mosaic_virus -	0	0	0	0	2	0	0	0	0	110
		Bacterial_spot -	Early_blight -	Late_blight -	Leaf_Mold -	Septoria_leaf_spot -	Spider_mites Two-spotted_spider_mite -	Target_Spot -	Yellow_Leaf_Curl_Virus -	healthy -	mosaic_virus -
		Predicted									

Figure 15: Confusion Matrix for EfficientNet-B0 (best model)

9 Conclusion & Future work

The primary objective of this project is to decrease food waste, a pressing concern in our daily lives. Inspired by the work of [1], we utilized VGG16 and several deep-learning techniques and were able to accomplish our goal of beating [1] in both efficacy and accuracy. We achieved an accuracy of 94.58%, which beats [1] by approximately 12.58%.

In addition to this achievement, another key objective of our project is to detect the level of healthiness of fruits and vegetables, specifically focusing on tomato disease classification. Our work, built on [2], utilized multiple models, including AlexNet, DenseNet, and EfficientNet. We achieved a notable accuracy of 96.04% using EfficientNet-B0, without synthetic images generated by C-GAN.

There will be a lot to do within the next few months. Future work will involve experimenting with different architectures, and further refining our methodology to maximize model accuracy. Furthermore, we plan to diversify and revamp our dataset by potentially looking for new fruits, vegetables, and potentially even foods to incorporate into our model. This project sets the stage for continued advancements in leveraging deep learning to address food waste.

As we continue to build and refine the C-GAN model, we aim to surpass the benchmark set by [2], striving for even higher accuracy in tomato leaf disease classification. This will also involve exploring other models to enhance our performance and applicability. Our ongoing efforts will contribute significantly to agricultural sustainability and food safety by providing more precise and reliable disease detection models.

Finally, we are excited to announce that we will be writing a conference paper which will be presented in Ankara with the help of Professor Elena in the upcoming months to share our findings and advancements with the scientific and academic community. This will allow us to spread our research, receive valuable feedback, and contribute to the ongoing dialogue in the field of deep learning and agricultural technology.

References

- [1] L. G. Fahad, S. F. Tahir, U. Rasheed, H. Saqib, M. Hassan, and H. Alquhayz, “Fruits and vegetables freshness categorization using deep learning,” *Computers, Materials & Continua*, vol. 71, no. 3, 2022.
- [2] J. S. G. M. Abbas, A. and S. Vankudothu, “Tomato plant disease detection using transfer learning with c-gan synthetic images,” *Computers and Electronics in Agriculture*, p. 106279, 2021.
- [3] “The expert center, the-food-waste-problem-and-how-to-reduce-it,” 2022. <https://ask-bioexpert.com/blog-post/the-food-waste-problem-and-how-to-reduce-it/>.
- [4] M. Davey, “Almost 90https://www.theguardian.com/environment/2017/oct/13/you-say-tomato-retailers-say-waste-research-finds-produce-problem,” Accessed May 21.
- [5] Wikipedia, “Neural network,” 2023. Accessed: December 26, 2023.
- [6] J. Brownle, “How do convolutional layers work in deep learning neural networks?,” 2020. <https://machinelearningmastery.com/convolutional-layers-for-deep-learning-neural-networks/>, Accessed: December 31, 2023.
- [7] “What is regularization? — IBM — ibm.com.” <https://www.ibm.com/topics/regularization>. [Accessed 01-01-2024].
- [8] K. Le, “An overview of vgg16 and nin models,” 2021. <https://medium.com/mllearning-ai/an-overview-of-vgg16-and-nin-models-96e4bf398484>.
- [9] T. Ahmed and N. H. N. Sabab, “Classification and understanding of cloud structures via satellite images with efficientunet,” *SN Computer Science*, vol. 3, no. 1, p. 99, 2022.
- [10] ‘school of public health. Harvard, “Food waste,” 2017. <https://www.hsph.harvard.edu/nutritionsource/sustainability/food-waste/>.
- [11] S. Stephanie, Greenly, “Global food waste in 2023,” 2022. <https://greenly.earth/en-us/blog/ecology-news/global-food-waste-in-2022>.

- [12] M. Quinet, T. Angosto, F. J. Yuste-Lisbona, R. Blanchard-Gros, S. Bigot, J.-P. Martinez, and S. Lutts, “Tomato fruit development and metabolism,” *Frontiers in plant science*, vol. 10, p. 475784, 2019.
- [13] S. Panno, S. Davino, A. G. Caruso, S. Bertacca, A. Crnogorac, A. Mandić, E. Noris, and S. Matić, “A review of the most common and economically important diseases that undermine the cultivation of tomato crop in the mediterranean basin,” *Agronomy*, vol. 11, no. 11, 2021.
- [14] D. Hughes, M. Salathé, *et al.*, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” *arXiv preprint arXiv:1511.08060*, 2015.
- [15] T. Göksu, Z. Kaya, and S. Sahmoud, “Classification of fruit images as fresh and rotten using convolutional neural networks,” in *2023 3rd International Conference on Computing and Information Technology (IC-CIT)*, pp. 297–301, IEEE, 2023.
- [16] U. Amin, M. I. Shahzad, A. Shahzad, M. Shahzad, U. Khan, and Z. Mahmood, “Automatic fruits freshness classification using cnn and transfer learning,” *Applied Sciences*, vol. 13, no. 14, p. 8087, 2023.
- [17] M. S. Miah, T. Tasnuva, M. Islam, M. Keya, M. R. Rahman, and S. A. Hossain, “An advanced method of identification fresh and rotten fruits using different convolutional neural networks,” in *2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pp. 1–7, IEEE, 2021.
- [18] M. Kulkarni, A. Chaudhari, S. Shegokar, K. Rudrawar, S. Joshi, S. Tibile, and I. Tayade, “Fruit freshness detection using cnn,” in *2022 International Conference on Futuristic Technologies (INCOFT)*, pp. 1–5, IEEE, 2022.
- [19] Y. Fu, M. Nguyen, and W. Q. Yan, “Grading methods for fruit freshness based on deep learning,” *SN Computer Science*, vol. 3, no. 4, p. 264, 2022.
- [20] M. Agarwal, A. Singh, S. Arjaria, A. Sinha, and S. Gupta, “Toled: Tomato leaf disease detection using convolution neural network,” *Procedia Computer Science*, vol. 167, pp. 293–301, 2020.

- [21] K. P. Ferentinos, “Deep learning models for plant disease detection and diagnosis,” *Computers and electronics in agriculture*, vol. 145, pp. 311–318, 2018.
- [22] L. Hailey, “19 tomato diseases: How to identify, treat, and prevent them,” 2023. <https://www.epicgardening.com/tomato-diseases/>, Accessed May 21.
- [23] J. A. Abbott, “Quality measurement of fruits and vegetables,” *Postharvest biology and technology*, vol. 15, no. 3, pp. 207–225, 1999.
- [24] “What are Neural Networks? — IBM — ibm.com.” <https://www.ibm.com/topics/neural-networks>. [Accessed 01-01-2024].
- [25] “What are Convolutional Neural Networks? — IBM — ibm.com.” <https://www.ibm.com/topics/convolutional-neural-networks>. [Accessed 01-01-2024].
- [26] <https://community.ibm.com/community/user/ai-datascience/people/pavan-saish-naru>, “Understanding Activations & Optimization- Neural Networks — community.ibm.com.” <https://community.ibm.com/community/user/ai-datascience/blogs/pavan-saish-naru/2023/01/27/optimization-hyperparameters>. [Accessed 01-01-2024].
- [27] “An Introduction to the ReLU Activation Function — builtin.com.” <https://builtin.com/machine-learning/relu-activation-function>. [Accessed 01-01-2024].
- [28] B. P. C, “Softmax Activation Function: Everything You Need to Know — Pinecone — pinecone.io.” <https://www.pinecone.io/learn/softmax-activation/>. [Accessed 02-01-2024].
- [29] “What is Gradient Descent? — IBM — ibm.com.” <https://www.ibm.com/topics/gradient-descent>. [Accessed 02-01-2024].
- [30] “Cross Entropy Loss: Intro, Applications, Code — v7labs.com.” <https://www.v7labs.com/blog/cross-entropy-loss-guide>. [Accessed 02-01-2024].
- [31] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.

- [32] Great.learning, “Everything you need to know about vgg16,” 2021. <https://medium.com/@mygreatlearning/everything-you-need-to-know-about-vgg16-7315defb5918>, , Accessed: December 28,2023.
- [33] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, “Densely connected convolutional networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017.
- [34] N. .AI, “Densenet,” 2023. <https://medium.com/nocoding-ai/densenet121-760df192f12d>, , Accessed: May 19,2024.
- [35] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Commun. ACM*, vol. 60, p. 84–90, may 2017.
- [36] S. Bangar, “Alexnet architecture explained,” 2022. <https://medium.com/@siddheshb008/alexnet-architecture-explained-b6240c528bd5>.
- [37] M. Tan and Q. V. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” 2020.
- [38] P. Potrimba, “What is efficientnet? the ultimate guide.,” 2023. <https://blog.roboflow.com/what-is-efficientnet/>.
- [39] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, “Going deeper with convolutions,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2015.
- [40] S. Kohli, “Understanding a classification report for your machine learning model,” 2019. <https://medium.com/@kohlishivam5522/understanding-a-classification-report-for-your-machine-learning-model-88815e2ce397>, Accessed: December 25,2023.